

Timetable AI Engine Logic

إليك **Timetable AI Engine Logic** بصياغة تشغيلية دقيقة، مناسبة لإرسالها مباشرة إلى Vibe Coding Agent أو وضعها داخل الـ PRD كنص تنفيذي واضح:

Timetable AI Engine Logic

الهدف من المحرك

بناء محرك ذكي لتوليد الجدول المدرسي داخل منصة نَسَق | NASSAQ بحيث يقوم بإنشاء جدول دراسي صالح ومنطقي ومرن، اعتمادًا بالكامل على بيانات قاعدة البيانات، مع احترام القيود الأكاديمية والتشغيلية، وتقليل التعارضات، وتحسين جودة التوزيع.

المحرك لا يجب أن يكون مجرد أداة تعبئة خانات، بل يجب أن يعمل كـ:

- Constraint-Aware Engine
- AI-Assisted Scheduler
- Dynamic Allocation System
- Versioned Timetable Generator

(1) الفكرة الأساسية للمحرك

المحرك يحاول حل المعادلة التالية لكل حصة مطلوبة:

فصل + مادة + معلم + وقت مناسب + بدون تعارض + ضمن النصاب + ضمن إعدادات المدرسة

أي أن كل حصة يتم إنشاؤها فقط إذا تحقق الآتي:

- الفصل متاح
- المادة مطلوبة لهذا الفصل
- يوجد معلم صالح لتدريسها
- الفترة الزمنية مناسبة ومسموح بها
- لا يوجد تعارض مع حصص أخرى
- نصاب المعلم لم يتجاوز الحدود
- لا يتم خرق قواعد اليوم الدراسي
- يتم احترام قواعد الجودة والتوزيع

(2) مصادر البيانات التي يعتمد عليها المحرك

يجب أن يسحب المحرك جميع البيانات من قاعدة البيانات، خاصة من:

- إعدادات المدرسة
- العام الدراسي
- الفصل الدراسي
- الأيام الدراسية النشطة
- الفترات الزمنية

- الحصص التعليمية فقط
- الاستراحات
- الصلاة
- الصفوف والفصول
- المواد
- عدد الحصص الأسبوعية لكل مادة
- ربط المواد بالصفوف
- ربط المعلمين بالمواد
- قيود توفر المعلمين
- قيود توفر الفصول
- الحدود القصوى للنصاب
- التفضيلات الزمنية
- النسخة الحالية من الجدول إن وجدت

لا يجوز استخدام أي قيم ثابتة داخل الواجهة أو الكود بشكل غير مرتبط بقاعدة البيانات.

(3) مراحل تشغيل المحرك

المرحلة الأولى: Readiness Check

قبل أي عملية توليد، يجب تنفيذ فحص جاهزية كامل.

يتأكد المحرك من:

- وجود مدرسة نشطة
- وجود عام دراسي نشط
- وجود فصل دراسي نشط
- وجود أيام أسبوع مفعلة
- وجود فترات زمنية صالحة
- وجود فترات تعليمية كافية
- وجود فصول دراسية
- وجود مواد مرتبطة بالصفوف
- وجود عدد حصص أسبوعي لكل مادة
- وجود معلم واحد على الأقل لكل مادة مطلوبة
- وجود إعدادات نصاب واضحة
- عدم وجود تناقضات تمنع التوليد

إذا فشل الفحص:

- لا يبدأ التوليد
- يعيد قائمة تفصيلية بالأخطاء
- يربط كل خطأ بالكيان المسبب له
- يعرض للمستخدم ما الذي يجب إصلاحه أولاً

المرحلة الثانية: Build Scheduling Context

بعد اجتياز الفحص، يقوم المحرك ببناء سياق التوليد.

يشمل ذلك:

- الأيام التعليمية الفعلية
- الفترات التعليمية فقط
- الفترات غير التعليمية المستثناة
- جميع الفصول المستهدفة
- جميع المواد المطلوبة لكل فصل
- جميع المعلمين المؤهلين لكل مادة
- جميع القيود الصلبة والمرنة
- جميع التفضيلات

الناتج من هذه المرحلة هو نموذج داخلي موحد يستخدمه المحرك أثناء الإسناد.

المرحلة الثالثة: Build Demand Units

يقوم المحرك بتحويل الاحتياج الأكاديمي إلى وحدات جدولة مستقلة.

مثال:

إذا كان فصل 5A يحتاج:

- رياضيات: 5 حصص أسبوعيًا
- علوم: 3 حصص أسبوعيًا
- عربي: 4 حصص أسبوعيًا

فالمحرك ينشئ:

- 5 وحدات جدولة للرياضيات
- 3 وحدات جدولة للعلوم
- 4 وحدات جدولة للعربي

كل وحدة تحتوي على:

- class_id
- grade_id
- subject_id
- candidate_teacher_ids
- weekly_index
- priority_weight
- constraints
- preferences

هذه الوحدات هي العناصر الفعلية التي يتم توزيعها داخل الجدول.

المرحلة الرابعة: Constraint Modeling

يجب أن يفصل المحرك بين نوعين من القيود:

أولاً: Hard Constraints

قيود لا يمكن كسرها إطلاقاً:

1. المعلم لا يمكن أن يدرّس حصتين في نفس الوقت
2. الفصل لا يمكن أن يحضر حصتين في نفس الوقت
3. لا يجوز وضع حصة داخل الاستراحة أو الصلاة
4. لا يجوز إسناد مادة لمعلم غير مربوط بها
5. لا يجوز تجاوز الحد الأقصى للحصص اليومية للمعلم
6. لا يجوز تجاوز الحد الأقصى للحصص الأسبوعية للمعلم
7. لا يجوز وضع الحصة في وقت غير متاح للمعلم
8. لا يجوز وضع الحصة في وقت غير متاح للفصل
9. يجب تحقيق عدد الحصص الأسبوعية المطلوبة لكل مادة
10. لا يجوز وضع حصة خارج حدود اليوم الدراسي
11. لا يجوز توليد خلية زمنية معطلة أو غير نشطة
12. لا يجوز تعيين غرفة غير مناسبة إذا كانت إدارة الغرف مفعلة

ثانيًا: Soft Constraints

يفضل احترامها قدر الإمكان لتحسين جودة الجدول:

1. توزيع المادة على أيام متعددة بدل تكديسها
2. تقليل الفجوات في جدول المعلم
3. تقليل الفجوات في جدول الفصل
4. تفضيل المواد الأساسية في أول اليوم
5. تجنب المواد الثقيلة في آخر اليوم
6. تقليل التكرار المتتالي لنفس المادة
7. تحقيق توازن في الحمل اليومي
8. احترام أوقات التفضيل الخاصة بالمعلم
9. ترك مرونة لبعض الفترات إن كانت المدرسة تريد ذلك

المرحلة الخامسة: Prioritization Engine

بعد بناء وحدات الجدولة، يقوم المحرك بترتيبها حسب **درجة الصعوبة**.

يجب أن يبدأ بالأصعب أولاً، مثل:

- مادة لها معلم واحد فقط
- مادة ذات حصص أسبوعية كثيرة
- مادة لها قيود زمنية صارمة
- فصل مزدحم بعدد كبير من المواد
- مادة أساسية
- مادة تحتاج غرفة خاصة
- معلم نصابه قريب من الحد الأقصى

الهدف:

تقليل احتمالية الفشل في المراحل اللاحقة.

المرحلة السادسة: Candidate Slot Generation

لكل وحدة جدول، يقوم المحرك بإنشاء قائمة بالفترات الزمنية المحتملة.

يتم استبعاد أي فترة إذا:

- المعلم مشغول
- الفصل مشغول
- الوقت غير متاح
- الفترة ليست Teaching Slot
- المادة ممنوعة في هذا التوقيت
- النصاب سيُكسر
- عدد مرات المادة في اليوم سيتجاوز الحد
- التوقيت خارج القواعد المحددة

النتائج هي مجموعة فترات مرشحة صالحة مبدئياً.

المرحلة السابعة: Slot Scoring Engine

كل فترة مرشحة يجب أن تحصل على درجة تقييم.

أمثلة لعناصر التقييم:

- المعلم متاح: 30+
- الفصل متاح: 30+
- الوقت مفضل للمادة: 15+
- الوقت مفضل للمعلم: 10+
- لا يسبب تكرار متتالي للمادة: 10+
- يحقق توازن الحمل اليومي: 10+
- يقلل الفجوات: 8+
- لا يضع مادة ثقيلة في آخر اليوم: 7+
- لا يكسب نفس المادة في نفس اليوم: 10+

ثم تُرتب الفترات من الأعلى إلى الأقل.

مبدأ مهم:

المحرك لا يختار أول خانة متاحة فقط، بل يختار أفضل خانة ممكنة.

المرحلة الثامنة: Iterative Assignment

يبدأ المحرك في الإسناد الفعلي وحدة بعد أخرى.

لكل وحدة:

1. يأخذ الوحدة ذات الأولوية الأعلى
2. يجلب قائمة الفترات المرشحة
3. يحسب Score لكل فترة
4. يختار أفضل فترة صالحة
5. ينشئ timetable_entry
6. يحدث حالة الإشغال للمعلم والفصل

7. يحدث عدادات النصاب
8. يحدث عدادات المادة اليومية والأسبوعية

ثم ينتقل إلى الوحدة التالية.

المرحلة التاسعة: Conflict Resolution

إذا تعذر إسناد وحدة في أي فترة مناسبة، يبدأ المحرك في حلول المعالجة.

الاستراتيجيات:

1. تجربة معلم آخر من المرشحين
2. تجربة ثاني أفضل فترة
3. Swap مع حصة أقل أولوية
4. Backtracking محدود
5. إعادة توزيع جزئي
6. Relax بعض الـ Soft Constraints فقط
7. تسجيل الوحدة كمشكلة unresolved إذا لم يوجد حل

ممنوع:

كسر Hard Constraints تحت أي ظرف.

المرحلة العاشرة: Optimization Pass

بعد اكتمال التوزيع الأولي، يبدأ المحرك مرحلة تحسين الجودة.

يفحص:

- الأيام المزدحمة
- الفجوات
- تكرار المادة
- توازن نصاب المعلمين
- توازن الأحمال بين الأيام
- كثافة المواد الأساسية
- جودة بداية ونهاية اليوم

يحاول:

- تبديل بعض الحصص
- نقل حصة من يوم مزدحم إلى يوم أخف
- تحسين تموضع مادة أساسية
- تقليل الفراغات
- تحسين النمط العام للجدول

دون كسر أي قيد إلزامي.

المرحلة الحادية عشرة: Quality Evaluation

بعد اكتمال التوزيع والتحسين، يحسب المحرك **درجة جودة نهائية**.

مثال على مكونات الجودة:

- اكتمال جميع الحصص المطلوبة
- صفر تعارضات إلزامية
- توازن حمل المعلمين
- توازن حمل الفصول
- تقليل الفجوات
- توزيع جيد للمواد
- احترام التفضيلات

مثال:

- completion_score
- hard_conflict_score
- teacher_balance_score
- class_balance_score
- subject_distribution_score
- soft_constraint_score

ثم ينتج:

- quality_score الإجمالي
- عدد التحذيرات
- عدد المشكلات
- نسبة اكتمال التوليد

المرحلة الثانية عشرة: Versioned Saving

يجب ألا يقوم المحرك بالحفظ فوق الجدول الحالي مباشرة.

يجب دعم:

- Draft Version
- Published Version
- Archived Versions

المنطق:

- كل توليد جديد ينشئ نسخة جديدة Draft
- يمكن للمستخدم مراجعتها
- يمكن نشرها لاحقاً
- يمكن أرشفة النسخ القديمة
- يمكن الرجوع لنسخة سابقة عند الحاجة

(4) أوضاع تشغيل المحرك

1. Full Generation

توليد جدول كامل من الصفر لجميع الفصول.

2. Partial Regeneration

إعادة توليد جزء فقط من الجدول، مثل:

- فصل معين
- معلم معين
- يوم معين
- مادة معينة

3. AI Assist Mode

تحليل الجدول الحالي واقتراح تحسينات دون إعادة التوليد الكامل.

(5) مخرجات المحرك

عند انتهاء التشغيل يجب أن يعيد المحرك:

- timetable_version_id
- generation_status
- generation_mode
- total_units
- assigned_units
- unassigned_units
- quality_score
- hard_conflicts_count
- soft_warnings_count
- affected_classes
- affected_teachers
- diagnostics_summary
- list of unresolved issues
- generation logs

(6) قواعد لا يمكن التنازل عنها

1. كل التوليد يجب أن يكون Database-Driven
2. لا تستخدم بيانات Static في الواجهة
3. لا تكسر Hard Constraints
4. لا تحفظ فوق النسخة المنشورة مباشرة
5. سجل كل عملية توليد وتعديل
6. إذا البيانات ناقصة، أوقف التوليد ووضح السبب
7. يجب أن يكون المحرك قابلاً للتوسع لاحقاً
8. يجب أن يدعم إعادة التوليد الجزئي

9. يجب أن يخرج Diagnostics واضحة
10. يجب أن يكون سريعًا ومنظمًا وقابلًا للصيانة

(7) الصيغة التنفيذية المختصرة للمطور

Timetable AI Engine يجب أن يعمل كالتالي:

- Read school context from database
- Validate readiness
- Build teaching demand units
- Build hard and soft constraints
- Prioritize units by difficulty
- Generate valid candidate slots
- Score candidate slots
- Assign best slot iteratively
- Resolve local conflicts using swap/backtracking
- Optimize final distribution
- Evaluate timetable quality
- Save as draft version
- Return diagnostics and summary

(8) Pseudocode تشغيلي

```
:def generate_timetable(school_id, academic_year_id, term_id, mode="full")
```

```
    context = load_timetable_context(school_id, academic_year_id, term_id)
```

```
        readiness = validate_context(context)
```

```
        :if not readiness.is_valid
```

```
            return build_readiness_error_response(readiness)
```

```
                demand_units = build_demand_units(context)
```

```
                demand_units = prioritize_units(demand_units, context)
```

```
                version = create_draft_version(context, mode)
```

```
                :for unit in demand_units
```

```

candidate_slots = build_candidate_slots(unit, context, version)
scored_slots = score_candidate_slots(unit, candidate_slots, context, version)

assigned = False

for slot in scored_slots:
    if passes_hard_constraints(unit, slot, context, version):
        assign_unit(unit, slot, version, context)
        assigned = True
        break

if not assigned:
    assigned = try_fallback_strategies(unit, version, context)

if not assigned:
    log_unresolved_issue(version, unit)

run_optimization_pass(version, context)

quality = evaluate_timetable_quality(version, context)
update_version_quality(version, quality)

return build_generation_response(version, quality)

```

9) الجملة الأساسية التي تحكم المنطق كله

الخوارزمية تحاول أن تجد لكل حصة مطلوبة:
 أفضل معلم صالح + أفضل وقت صالح + أفضل توزيع ممكن + بدون أي تعارض + مع احترام النصاب
 والقيود الأكاديمية

